ASSIGNMENT 8.3

NAME: T. Prashanthi

2303A52495

BATCH:50

Task 1: Email Validation using TDD

Scenario

You are developing a user registration system that requires reliable email input validation.

Requirements

• Must contain @ and . characters

• Must not start or end with special characters

• Should not allow multiple @ symbols

• AI should generate test cases covering valid and invalid email formats

• Implement is_valid_email(email) to pass all AI-generated test cases

Expected Output

• Python function for email validation

• All AI-generated test cases pass successfully

• Invalid email formats are correctly rejected

• Valid email formats return True

PROMPT:

valid_emails=["prashanthi@gmail.com","prashanthi.thirunahari@gmail.com","prashanthi123@gmail.com","p_thirunahari@gamil.com","prashanthi.thirunahari@edu.in"]

CODE:

```
#2303A53495

# valid_emails=["prashanthi@gmail.com","prashanthi.thirunahari@gmail.com","prashanthi123@gmail.com","p_thirunahari@gamil.com"

import re
def validate_email(email):
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    if re.match(pattern, email):
        return True
    else:
        return False
valid_emails = [
    "prashanthi@gmail.com",
    "prashanthi.thirunahari@gmail.com",
    "prashanthi123@gmail.com",
    "p_thirunahari@gamil.com",
    "prashanthi.thirunahari@edu.in"
]

for email in valid_emails:
    if validate_email(email):
        print(f"{email} is valid.")
    else:
        print(f"{email} is invalid.")
```

OUTPUT:

```
prashanthi.thirunahari@gmail.com is valid.
prashanthi123@gmail.com is valid.
p_thirunahari@gamil.com is valid.
prashanthi.thirunahari@edu.in is valid.
PS C:\Users\prash\OneDrive\Desktop>
```

EXPLANATION:

To implement email validation using Test-Driven Development (TDD). First, we create test cases for both valid and invalid email formats based on the given rules. Then, we write the is_valid_email() function so that it returns True for valid emails and False for invalid ones. This ensures reliable and accurate email verification in the user registration system.

Task 2: Grade Assignment using Loops

Scenario

You are building an automated grading system for an online examination platform.

Requirements

• AI should generate test cases for assign_grade(score) where:

– 90–100 → A

– 80–89 → B

– 70–79 → C

– 60–69 → D

– Below 60 → F

• Include boundary values (60, 70, 80, 90)

• Include invalid inputs such as -5, 105, "eighty"

• Implement the function using a test-driven approach

Expected Output

• Grade assignment function implemented in Python

• Boundary values handled correctly

• Invalid inputs handled gracefully

• All AI-generated test cases pass

PROMPT:

develop function in python assign_grade(score) using test driven development which , returns A for score 90 and 100, B for score 80 and 89, C for score 70 and 79, D for score 60 and 69,F for score below 60. Write test cases for the function before implementing it.handle boundary values(60,70,80,90).handles invalid inputs like negative numbers,values above 100,and non numeric inputs .

CODE:

```python
# develop function in python assign_grade(score) using test driven development which
# returns A for score 90 and 100, B for score 80 and 89, C for score 70 and 79, D for score 60 and 69,
#  F for score below 60. Write test cases for the function before implementing it.
#  handle boundary values(60,70,80,90).handles invalid inputs like negative numbers,values above 100,
# and non numeric inputs .

def assign_grade(score):
    if not isinstance(score, (int, float)):
        return "Invalid input: Score must be a number."
    if score < 0 or score > 100:
        return "Invalid input: Score must be between 0 and 100."
    if score >= 90:
        return 'A'
    elif score >= 80:
        return 'B'
    elif score >= 70:
        return 'C'
    elif score >= 60:
        return 'D'
    else:
        return 'F'
# Test cases
print(assign_grade(95))
print(assign_grade(85))
print(assign_grade(75))
print(assign_grade(65))
print(assign_grade(55))
print(assign_grade(60))
print(assign_grade(70))
print(assign_grade(80))
print(assign_grade(90))
print(assign_grade(-5))
print(assign_grade(105))
print(assign_grade("A+"))
```

OUTPUT:

```
A
B
C
D
F
D
C
B
A
Invalid input: Score must be between 0 and 100.
Invalid input: Score must be between 0 and 100.
Invalid input: Score must be a number.
```

EXPLANATION:

This program assigns grades based on score ranges using conditional statements. Boundary values like 60, 70, 80, and 90 are correctly handled using ≥ conditions. Invalid inputs such as negative numbers, values above 100, or non-numeric data are checked using type and range validation. Using TDD ensures all test cases pass before finalizing the function.

Task 3: Sentence Palindrome Checker

Scenario

You are developing a text-processing utility to analyze sentences.

Requirements

• AI should generate test cases for is_sentence_palindrome(sentence)

• Ignore case, spaces, and punctuation

• Test both palindromic and non-palindromic sentences

• Example:

– "A man a plan a canal Panama" → True

Expected Output

• Function correctly identifies sentence palindromes

• Case and punctuation are ignored

• Returns True or False accurately

• All AI-generated test cases pass

PROMPT:

Develop a text processing function is_sentence_palindrome(sentence). Check wether a sentence is a palindrome.Ignores case differences.Ignores spaces.Ignores punctuation.returns true if the sentence is a palindrome.return false otherwise.

CODE:

```python
import re
def is_sentence_palindrome(sentence):
    cleaned_sentence = re.sub(r'[^A-Za-z0-9]', '', sentence).lower()
    return cleaned_sentence == cleaned_sentence[::-1]
palindrome_cases = [
    "A man a plan a canal Panama",

]
non_palindrome_cases = [
    "Hello World",

]
for sentence in palindrome_cases:
    if is_sentence_palindrome(sentence):
        print(f'"{sentence}" is a palindrome.')
    else:
        print(f'"{sentence}" is not a palindrome.')
for sentence in non_palindrome_cases:
    if is_sentence_palindrome(sentence):
        print(f'"{sentence}" is a palindrome.')
    else:
        print(f'"{sentence}" is not a palindrome.')
```

OUTPUT:

```
"A man a plan a canal Panama" is a palindrome.
"Hello World" is not a palindrome.
```

EXPLANATION:

This program checks whether a sentence is a palindrome by first removing spaces, punctuation, and converting all characters to lowercase. It then compares the cleaned sentence with its reverse. If both are equal, the function returns True, otherwise False. Using TDD ensures both palindromic and non-palindromic cases are properly tested.

Task 4: ShoppingCart Class

Scenario

You are designing a basic shopping cart module for an e-commerce application.

Requirements

• AI should generate test cases for the ShoppingCart class

• Class must include the following methods:

– add_item(name, price)

– remove_item(name)

– total_cost()

• Validate correct addition, removal, and cost calculation

• Handle empty cart scenarios

Expected Output

• Fully implemented ShoppingCart class

• All methods pass AI-generated test cases

• Total cost is calculated accurately

• Items are added and removed correctly

PROMPT:

Generate a python function to develop a shopping cart class for an e-commerce application using a test-driven approach.That class must contains Add items using add_items(name,price).Remove items using remove_item(name).Total price using total_cost().Validate correct addition, removal, and cost calculation.Handle empty cart scenarios.

CODE:

```python
class ShoppingCart:
    def __init__(self):
        self.items = {}
    def add_item(self, name, price):
        self.items[name] = price
    def remove_item(self, name):
        if name in self.items:
            del self.items[name]
    def total_cost(self):
        return sum(self.items.values())
# Test cases
cart = ShoppingCart()
cart.add_item("Laptop", 1000)
cart.add_item("Phone", 500)
print(cart.total_cost())
cart.remove_item("Phone")
print(cart.total_cost())
cart.remove_item("Tablet")
print(cart.total_cost())
cart.remove_item("Laptop")
print(cart.total_cost())
```

OUTPUT:

```
1500
1000
1000
0
```

EXPLANATION:

The ShoppingCart class stores items in a dictionary where the key is the item name and the value is its price. The add_item() method adds products, remove_item() deletes them if they exist, and total_cost() calculates the total price using the sum of all values. The program also handles empty cart cases and invalid removals gracefully. Using TDD ensures all operations work correctly before final implementation.

Task 5: Date Format Conversion

Scenario

You are creating a utility function to convert date formats for reports.

Requirements

• AI should generate test cases for convert_date_format(date_str)

• Input format must be "YYYY-MM-DD"

• Output format must be "DD-MM-YYYY"

• Example:

– "2023-10-15" → "15-10-2023"

Expected Output

• Date conversion function implemented in Python

• Correct format conversion for all valid inputs

• All AI-generated test cases pass successfully

PROMPT:

Generate a python function Develop a function convert_date_format(date_str) using a test-driven approach that: Accepts date in "YYYY-MM-DD" format. Converts it into "DD-MM-YYYY" format. Returns the converted date string. Ensures all valid inputs are correctly transformed.

CODE:

```python
def convert_date_format(date_str):
    try:
        year, month, day = date_str.split('-')
        return f"{day}-{month}-{year}"
    except ValueError:
        return "Invalid date format. Please use 'YYYY-MM-DD'."
# Test cases
print(convert_date_format("2024-06-15"))
print(convert_date_format("2004-01-01"))
```

OUTPUT:

```
15-06-2024
01-01-2004
PS C:\Users\prash\OneDrive\Desktop>
```

EXPLANATION:

The function splits the input date string using "-" and rearranges the parts from "YYYY-MM-DD" to "DD-MM-YYYY". It also checks whether the input follows the correct format and contains numeric values. If the format is incorrect, it returns "Invalid Format". Using TDD ensures all valid and invalid cases are properly tested before finalizing the function.